LAPORAN PRAKTIKUM

ALGORITMA PEMROGRAMAN 2

MODUL 5 – REKURSIF

NAMA: Dinar Azzahra Putri Setiadi

NIM: 109082500176

KELAS S1IF-13-06

PROGRAM STUDI TEKNIK INFORMATIKA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

Rekursif dalam pemrograman adalah cara menyelesaikan masalah dengan membuat fungsi memanggil dirinya sendiri berkali-kali sampai mencapai kondisi awal yang menghentikan proses tersebut. Konsep ini sangat membantu untuk menyelesaikan masalah yang bisa dibagi menjadi beberapa sub-masalah lebih kecil dengan pola yang sama, misalnya perhitungan faktorial, deret Fibonacci, dan pencarian dalam struktur data seperti pohon. Dalam penerapannya, rekursif perlu memiliki dua bagian penting, yaitu kondisi dasar yang berfungsi untuk menghentikan proses dan langkah rekursif yang bertugas untuk mengulang proses tersebut. Meskipun menggunakan rekursif bisa membuat kode lebih pendek dan lebih mudah dimengerti, kita juga harus waspada karena metode ini bisa membuat penggunaan memori lebih besar karena fungsi dipanggil berulang kali.

B. Guided

1. [Baris Bilangan]

```
package main

import "fmt"

func main() {
    var n int
    fmt.Scan(&n)
    baris(n)
}
func baris(bilangan int) {
    if bilangan == 1 {
        fmt.Println(1)
    } else {
        fmt.Println(bilangan)
        baris(bilangan - 1)
    }
}
```

Output :

```
PROBLEMS    OUTPUT    TERMINAL    DEBUG CONSOLE    PORTS

PS D:\ALPRO 2\109082500176_Dinar-Azzahra-Putri-Setiadi\Week 4 - Modul 5\Guided-1> go run barisbi
langan.go
4
4
3
2
1
PS D:\ALPRO 2\109082500176_Dinar-Azzahra-Putri-Setiadi\Week 4 - Modul 5\Guided-1>
```

2. [Penjumlahan]

```go
package main

import "fmt"

func main() {
    var n int
    fmt.Scan(&n)
    fmt.Println(penjumlahan(n))
}

func penjumlahan(n int) int {
    if n == 1 {
        return 1
    } else {
        return n + penjumlahan(n-1)
    }
}
```

Output :

```
PROBLEMS    OUTPUT    TERMINAL    DEBUG CONSOLE    PORTS

PS D:\ALPRO 2\109082500176_Dinar-Azzahra-Putri-Setiadi\Week 4 - Modul 5\Guided-2> go run penjuml
ahan.go
6
21
PS D:\ALPRO 2\109082500176_Dinar-Azzahra-Putri-Setiadi\Week 4 - Modul 5\Guided-2>
```

3. [Dua Pangkat N]

```go
package main

import "fmt"

func main() {
    var n int
    fmt.Scan(&n)
    fmt.Println(pangkat(n))
}

func pangkat(n int) int {
    if n == 0 {
        return 1
    } else {
        return 2 * pangkat(n-1)
    }
}
```

Output :

```
PROBLEMS    OUTPUT    TERMINAL    DEBUG CONSOLE    PORTS

PS D:\ALPRO 2\109082500176_Dinar-Azzahra-Putri-Setiadi\Week 4 - Modul 5\Guided-3> go run duapang
katn.go
6
64
PS D:\ALPRO 2\109082500176_Dinar-Azzahra-Putri-Setiadi\Week 4 - Modul 5\Guided-3>
```

C. Unguided

1. Soal Unguided 1

Deret fibonacci adalah sebuah deret dengan nilai suku ke-0 dan ke-1 adalah 0 dan 1, dan nilai suku ke-n selanjutnya adalah hasil penjumlahan dua suku sebelumnya. Secara umum dapat diformulasikan $\cdot\ \cdot = \cdot\ \cdot_{-1} + \cdot\ \cdot_{-2}$. Berikut ini adalah contoh nilai deret fibonacci hingga suku ke-10. Buatlah program yang mengimplementasikan fungsi rekursif pada deret Fibonacci tersebut.

Jawaban :

```go
package main

import "fmt"

func fibonacci(n int) int {
    if n == 0 {
        return 0
    } else if n == 1 {
        return 1
    } else {
        return fibonacci(n-1) + fibonacci(n-2)
    }
}

func main() {
    var n int = 10

    fmt.Println("Deret Fibonacci sampai suku ke-", n,
":")

    for i := 0; i <= n; i++ {
        fmt.Print(fibonacci(i), " ")
    }
}
```

Output :

```
PROBLEMS 1   OUTPUT   TERMINAL   DEBUG CONSOLE   PORTS

PS D:\ALPRO 2\109082500176_Dinar-Azzahra-Putri-Setiadi\Week 4 - Modul 5\Unguided-1> go run fibon
acci.go
Deret Fibonacci sampai suku ke- 10 :
0 1 1 2 3 5 8 13 21 34 55
PS D:\ALPRO 2\109082500176_Dinar-Azzahra-Putri-Setiadi\Week 4 - Modul 5\Unguided-1>
```

Penjelasan :

Program ini memakai fungsi rekursif yang disebut fibonacci untuk menghitung angka dalam deret Fibonacci. Fungsi ini memiliki aturan awal, yaitu jika n sama dengan 0 maka mengembalikan nilai 0, dan jika n sama dengan 1 maka mengembalikan nilai 1. Jika nilai n lebih dari 1, fungsi akan memanggil dirinya sendiri dengan rumus fibonacci(n-1) ditambah fibonacci(n-2) sesuai dengan definisi deret Fibonacci. Dalam fungsi utama, program menentukan hingga mana batas suku Fibonacci yang akan ditampilkan, yaitu

sampai 10, lalu menggunakan perulangan for untuk memanggil fungsi Fibonacci mulai dari indeks 0 sampai 10, serta mencetak hasilnya ke layar.

2. Soal Unguided 2

Buatlah program yang mengimplementasikan rekursif untuk menampilkan faktor bilangan dari suatu N, atau bilangan yang apa saja yang habis membagi N. **Masukan** terdiri dari sebuah bilangan bulat positif N. **Keluaran** terdiri dari barisan bilangan yang menjadi faktor dari N (terurut dari 1 hingga N ya).

Jawaban :

```
package main

import "fmt"

func faktor(n int, i int) {
    if i > n {
        return
    }
    if n%i == 0 {
        fmt.Print(i, " ")
    }
    faktor(n, i+1)
}

func main() {
    var n int
    fmt.Print("Masukkan bilangan: ")
    fmt.Scan(&n)

    fmt.Print("Faktor dari ", n, " adalah: ")
    faktor(n, 1)
```

Output :

```
PROBLEMS 1   OUTPUT   TERMINAL   DEBUG CONSOLE   PORTS

PS D:\ALPRO 2\109082500176_Dinar-Azzahra-Putri-Setiadi\Week 4 - Modul 5\Unguided-2> go run fakto
rbilangann.go
Masukkan bilangan: 12
Faktor dari 12 adalah: 1 2 3 4 6 12
PS D:\ALPRO 2\109082500176_Dinar-Azzahra-Putri-Setiadi\Week 4 - Modul 5\Unguided-2> go run fakto
rbilangann.go
Masukkan bilangan: 5
Faktor dari 5 adalah: 1 5
PS D:\ALPRO 2\109082500176_Dinar-Azzahra-Putri-Setiadi\Week 4 - Modul 5\Unguided-2>
```

Penjelasan :

Program ini memakai fungsi faktor rekursif untuk memeriksa setiap bilangan dari 1 hingga N. Jika angka itu bisa membagi N, maka akan dicetak sebagai faktor. Proses diulang terus menerus dengan memanggil fungsi itu sendiri sampai mencapai batas N. Saat program dijalankan, pengguna diminta memasukkan angka N, kemudian program akan menampilkan semua faktor dari N secara berurutan mulai dari 1 sampai N. Contohnya, jika inputnya adalah 12, maka outputnya akan menjadi: 1 2 3 4 6 12.

3. Soal Unguided 3

Buatlah program yang mengimplementasikan rekursif untuk menampilkan barisan bilangan ganjil. **Masukan** terdiri dari sebuah bilangan bulat positif N. **Keluaran** terdiri dari barisan bilangan ganjil dari 1 hingga N.

Jawaban :

```
package main

import "fmt"

func ganjil(n int, i int) {
    if i > n {
        return
    }
    if i%2 != 0 {
        fmt.Print(i, " ")
    }
    ganjil(n, i+1)
}

func main() {
    var n int
    fmt.Print("Masukkan bilangan: ")
    fmt.Scan(&n)

    fmt.Print("Bilangan ganjil dari 1 sampai ", n, "
adalah: ")
    ganjil(n, 1)
}
```

Output :

```
PROBLEMS 1   OUTPUT   TERMINAL   DEBUG CONSOLE   PORTS

PS D:\ALPRO 2\109082500176_Dinar-Azzahra-Putri-Setiadi\Week 4 - Modul 5\Unguided-3> go run bilan
ganganjil.go
Masukkan bilangan: 5
Bilangan ganjil dari 1 sampai 5 adalah: 1 3 5
PS D:\ALPRO 2\109082500176_Dinar-Azzahra-Putri-Setiadi\Week 4 - Modul 5\Unguided-3> go run bilan
ganganjil.go
Masukkan bilangan: 20
Bilangan ganjil dari 1 sampai 20 adalah: 1 3 5 7 9 11 13 15 17 19
PS D:\ALPRO 2\109082500176_Dinar-Azzahra-Putri-Setiadi\Week 4 - Modul 5\Unguided-3>
```

Penjelasan :

Program ini menggunakan fungsi rekursif untuk memeriksa setiap bilangan dari 1 sampai N. Jika angka tersebut ganjil (tidak bisa dibagi 2), maka akan ditampilkan, kemudian fungsi tersebut akan memanggil dirinya sendiri sampai mencapai N. Saat program dijalankan, pengguna diminta untuk memasukkan nilai N, lalu program akan menampilkan urutan bilangan ganjil mulai dari 1 hingga mencapai N. Contohnya, jika masukan adalah 20, maka hasilnya adalah: 1 3 5 7 9 11 13 15 17 19.

D. Kesimpulan

Kesimpulan dari praktikum ini yaitu, dapat disimpulkan bahwa fungsi rekursif adalah cara yang efektif untuk menyelesaikan masalah yang memiliki pola berulang, seperti deret Fibonacci, faktor bilangan, dan barisan bilangan ganjil. Dengan memanggil fungsi itu sendiri, program jadi lebih mudah dan rapi. Namun, dalam menggunakan program tersebut, penting untuk memperhatikan kondisi dasar agar tidak terjadi perulangan yang tidak berhenti. Selain itu, meskipun cara rekursif membuat kode lebih singkat,

penggunaan memori biasanya lebih besar dibandingkan metode iteratif, jadi cara ini harus digunakan dengan bijak sesuai dengan kebutuhan.